

Recocido simulado por aceptación de umbrales para una variante del problema del agente viajero

Lara Guillén Gilabert

En colaboración el grupo del Seminario de Heurísticas de Optimización Combinatoria. Quiero dar mis más sincero agradecimiento a las personas que durante el desarrollo de este proyecto intercambiaron conmigo sus ideas y resultados, en especial a Wendy Sánchez Cortés, Manuel Ian-luck Rojo Peña y Carlos Cabrera Ramírez. Este último fue quien tuvo la generosidad de echarme una mano con la generación de mapas que se encuentran en este documento.

Abstract—En este trabajo se presenta el diseño, implementación y evaluación experimental de un programa en orientado a la resolución de la variante del Problema del Agente Viajero sobre caminos hamiltonianos mediante la heurística de recocido simulado por aceptación de umbrales (Threshold Accepting, Dueck & Scheuer). La implementación escrita en Java evalúa el costo de cada solución de forma eficiente y permite reproducir cualquier ejecución a partir de su semilla. Se mejoró la forma en que el algoritmo explora soluciones vecinas, permitiendo explorar el espacio de búsqueda de forma más precisa.

1. INTRODUCCIÓN

La optimización combinatoria sobre gráficas constituye uno de los núcleos teóricos más importantes en las ciencias de la computación, debido a sus aplicaciones prácticas. El Problema del Agente Viajero (TSP, Traveler Salesman Problem por sus siglas en inglés) destaca por ser uno de los problemas NP-difícil más estudiados en el área, cuyas variantes de optimización sobre gráficas ponderadas generales resultan intratables mediante algoritmos exactos. Cuenta con aplicaciones directas en áreas como en logística y reparto para establecer rutas óptimas, en genómica para la secuenciación de ADN o en manufactura de circuitos para decidir el recorrido que realizan las máquinas que los taladran. Por ello, el diseño de heurísticas y métodos estocásticos se vuelve imperativo.

En este trabajo se propone e implementa una heurística de optimización combinatoria basada en Aceptación por Umbrales (Threshold Accepting, TA), originalmente desarrollada por Dueck y Scheuer [dueck1990threshold]. El algoritmo que se presenta es una variante determinista del recocido simulado que resuelve la variante del Problema del Agente Viajero sobre caminos hamiltonianos: dado un conjunto de ciudades y las distancias entre ellas, se busca la ruta de menor costo que las visite todas. El tamaño de búsqueda para esta variante respecto a n ciudades es de $\frac{n!}{2}$ caminos distintos posibles, sin distinción de dirección.

La idea del algoritmo se basa en la optimización local: se parte de una ruta inicial y se proponen pequeños cambios locales, aceptando aquellos que no empeoren significativamente el costo actual al comenzar con umbrales altos y reducirlos gradualmente. Esto divide al algoritmo en una etapa de exploración del espacio de soluciones, donde frecuentemente se aceptan rutas peores, y una etapa de explotación, donde la exigencia aumenta hasta converger a una solución de bajo costo. No se puede asegurar que la solución encontrada es la óptima; se busca que sea lo suficientemente buena y pueda calcularse en pocos minutos o segundos.

2. PROBLEMA DEL AGENTE VIAJERO SOBRE CAMINOS HAMILTONIANOS

labelsec:problema

2.1. Formalización del espacio de búsqueda

Sea S el conjunto de k ciudades que forman parte de la instancia a tratar, y sea $G = (S, E)$ la gráfica que S induce a partir de la

base de datos disponible. Cada arista $(u, v) \in E$ posee un peso asociado $w(u, v) > 0$, correspondiente a la distancia real de traslado entre ambas ciudades. Dado que no todo par de vértices cuenta con una conexión directa, G es, en el caso general, una gráfica no completa.

Una solución candidata P se define como una permutación de los elementos de S ; es decir, una secuencia $(P_1, P_2, \dots, P_k)$ que visita cada ciudad exactamente una vez. Al tratarse de una variante sobre caminos hamiltonianos, nunca se regresa al vértice de origen. El costo se evalúa únicamente a lo largo de las $k - 1$ conexiones consecutivas (P_i, P_{i+1}) .

2.2. Métrica de distancia y función de costo penalizada

Para la evaluación de una permutación sobre una gráfica, el modelo se basa en las siguientes definiciones:

- **Distancia natural**, $d(u, v)$: Distancia geográfica entre dos ciudades calculada mediante la fórmula *Haversine* a partir de sus coordenadas. Esta función se puede calcular independientemente de si la arista (u, v) pertenece o no a E .
- **Distancia máxima**, $\max_d(S)$: Cota superior de los pesos asociados a las aristas válidas de la instancia:

$$\max_d(S) = \max_{(u,v) \in E} w(u, v) \quad (1)$$
- **Peso aumentado**, $w_S(u, v)$: Penalización estricta por conexión ausente entre ciudades:

$$w_S(u, v) = \begin{cases} w(u, v) & \text{si } (u, v) \in E \\ d(u, v) \times \max_d(S) & \text{si } (u, v) \notin E \end{cases} \quad (2)$$

Cuando la arista existe en G , se toma su peso real; de lo contrario, se impone un castigo al multiplicar su distancia natural por el valor máximo presente en la instancia. Esta penalización supera ampliamente cualquier distancia admisible, superando por varios órdenes de magnitud el valor de las distancias reales. Gracias a esta penalización es que se puede discernir fácilmente soluciones factibles de aquellas que no lo son.

- **Factor de normalización**, $N(S)$: Suma de los $k - 1$ pesos válidos más grandes presentes en G . Permite comparar el comportamiento del algoritmo sobre instancias de distinta cardinalidad.
- **Costo normalizado**, $f(P)$: La función objetivo a minimizar queda formulada como:

$$f(P) = \frac{1}{N(S)} \sum_{i=1}^{k-1} w_S(P_i, P_{i+1}) \quad (3)$$

2.3. Criterio analítico de factibilidad

Una permutación P se clasifica como **factible** si representa un camino hamiltoniano válido en G ; es decir, si cada transición consecutiva satisface $(P_i, P_{i+1}) \in E$. Por la construcción de $N(S)$ y la severidad

del peso aumentado w_s , la factibilidad de una ruta se determina de manera exacta y sin costo computacional adicional a partir de su evaluación numérica:

$$f(P) \leq 1 \iff P \text{ es factible} \quad (4)$$

Si una permutación contiene al menos una arista ausente, el término de penalización correspondiente sobrepasa individualmente el valor de $N(S)$, provocando que $f(P) > 1$.

El espacio de búsqueda comprende $\frac{n!}{2}$ rutas posibles, donde n es el número de ciudades de la instancia a tratar. Cuando n crece, la evaluación exhaustiva se vuelve inviable. Para guiar la optimización local, se introduce la noción de solución vecina: una permutación P' derivada de P mediante el intercambio de un par de aristas. Por tanto, lo que se busca es encontrar una permutación que minimice $f(P)$, que por supuesto sea una trayectoria factible en G .

3. HEURÍSTICA: RECOCIDO SIMULADO DE ACEPTACIÓN POR UMBRALES

Dado el crecimiento factorial del espacio de soluciones, es natural buscar una heurística que sea capaz de aproximarnos a una buena solución; una que minimice el costo. En metalurgia, el recocido (*annealing*) es una técnica de tratamiento técnico aplicada a metales: se calienta por encima de una temperatura crítica para luego dejarse enfriar de manera lenta y controlada. Cuando la temperatura es alta, las partículas del material poseen una energía cinética muy alta que les permite moverse de manera más libre por el espacio. Conforme la temperatura desciende, la movilidad se ve reducida y los átomos se ven obligados a organizarse en estados de menor energía, haciendo que el sistema termine con configuraciones estables de energía mínima global.

Scott Kirkpatrick, C. Daniel Gelatt y Mario Vecchi retomaron la idea del recocido en metalurgia para plantear un análogo donde también se busca encontrar la configuración de costo mínimo. Propusieron sustituir la función de energía del sistema físico por la función objetivo de costo, y parametrizar el proceso con una temperatura de control [kirkpatrick1983optimization]. Más adelante, Dueck y Scheuer cuestionaron el cálculo probabilístico para escapar de mínimos locales. Propusieron la Aceptación por Umbrales como alternativa [dueck1990threshold] con una regla de aceptación determinista:

$$\text{aceptar } s' \iff f(s') \leq f(s) + T,$$

3.1. Búsqueda local

En cada iteración se genera un vecino s' de la solución actual s mediante una modificación local mínima del camino, y se acepta si su costo no excede el de s en más que el umbral T actual. T puede iniciar siendo alta, permitiendo aceptar movimientos que empeoran la solución, lo que evita quedar atrapado en mínimos locales desde temprano. Se parte desde T_0 y se reduce mediante un factor de enfriamiento $\varphi \in (0, 1)$, que suele ser cercano a 1 para un enfriamiento más lento.

3.2. Vecinidad

El operador de vecindad usado inicialmente intercambiaba dos posiciones del camino elegidas al azar, alterando hasta cuatro aristas por movimiento. Se sustituyó por una reversión de segmento [croes1958method]: se eligen dos índices $i < j$ al azar y se invierte el subcamino entre ellos, lo que altera a lo más dos aristas, que son esencialmente las que conectan el segmento invertido con el resto del camino. Las aristas internas del segmento son las mismas recorridas en sentido contrario y, al ser la matriz de pesos de G

simétrica, su suma no cambia. Un cruce en la ruta siempre es subóptimo bajo una métrica geométrica, y la reversión lo elimina en un solo paso, mientras que el intercambio solo llega a la misma configuración pasando por estados intermedios que el umbral puede rechazar.

3.3. Convergencia y terminación

El algoritmo no garantiza optimalidad. Termina cuando $T \leq \varepsilon$. Para evitar que una combinación de parámetros que no llega a enfriarse deje el programa corriendo indefinidamente, también se introdujeron un número máximo de intentos. El resultado reportado es la mejor solución vista a lo largo de toda la ejecución.

4. DISEÑO DE LA SOLUCIÓN

El sistema se organiza en cuatro capas con dependencias en un solo sentido: `data` (lectura de la base de datos), `domain` (la instancia y las soluciones), `heuristic` (el algoritmo) y `experiment` (los barridos de semillas), coordinadas por `app.Main`.

4.1. Arquitectura del proyecto

El proyecto se organiza en seis paquetes bajo `mx.unam.fciencias.tsp`:

```
mx.unam.fciencias.tsp
|-- data      DatabaseBuilder, GraphDao, InstanceReader,
|             DataException
|-- domain    HaversineDistance, Instance, Solution,
|             Route, CostTrace, InvalidInstanceException,
|             InvalidRouteException
|-- exhaustive Permutation
|-- heuristic  SimulatedAnnealing, Parameters, Outcome,
|             StopReason, CostListener
|-- experiment SeedSweep, Trial, TrialWriter
|-- app       Main, Configuration, PropertiesReader,
|             ParameterException
```

4.2. Estructuras de Datos Principales

Una instancia mantiene dos matrices $k \times k$: `weights`, con el peso real o 0.0 si la arista está ausente, y `augmented`, con $w_s(u, v)$ precalculada. Mantener ambas evita recalcular la distancia Haversine de cada arista ausente en cada evaluación de costo.

El mapeo de identificador de ciudad a posición dentro de la instancia se resuelve con un arreglo indexado por `id-1`.

Una solución se representa como un arreglo de k enteros, que es una permutación de posiciones, junto con su costo ya calculado, para no tener que recalcularlo en cada consulta.

4.3. Flujo del programa

Una ejecución atraviesa los paquetes en este orden:

1. `app.Main` lee la configuración (`Configuration`, `PropertiesReader`) desde un `.properties`.
2. `data.DatabaseBuilder` construye el `.db` a partir del `.sql` si aún no existe, y lo reutiliza en caso contrario.
3. `data.InstanceReader` lee el `.tsp` y produce los `id`'s de S en el orden del archivo.
4. `data.GraphDao` traduce esos `ids` a posiciones y entrega la subgráfica inducida por S : la matriz de pesos y las coordenadas.
5. Con esos arreglos se construye una `domain.Instance`: valida sus invariantes, y precalcula `maxWeight()`, `normalizer()` y la matriz de pesos aumentados.

6. A partir de la `Instance` se resuelve el problema por uno de dos caminos, mutuamente ajenos:
 - `exhaustive.Permutation` enumera las $k!$ ordenaciones y devuelve la óptima, solo viable para k pequeña.
 - `heuristic.SimulatedAnnealing.bestRoute` ejecuta los procedimientos 1 a 5 (lote, corrida por umbrales, búsqueda de temperatura inicial) sobre objetos `Route`, guiada por `Parameters`, y devuelve un `Outcome` con la mejor `Route` vista, su costo y el `StopReason`.
7. Para experimentación, `experiment.SeedSweep` repite el paso anterior una vez por semilla sobre la misma `Instance`, en paralelo, y `TrialWriter` acumula los pares (`seed`, `cost`) en un CSV.
8. `Main` imprime el camino resultante (traducido de vuelta a ids de ciudad) y, en modo verboso, la traza de costos vía `CostListener`.

4.4. Procedimientos de la heurística

El recocido simulado se divide en cinco procedimientos: el lote (`batch`), la ejecución por umbrales (`bestRoute`), la búsqueda de temperatura inicial (`temperatureSearch`), el porcentaje de aceptación (`countsAccepted`) y la búsqueda binaria (`binaryTemperatureSearch`) [pelaez-notas].

4.4.1. Lotes

Genera vecinos hasta aceptar L soluciones o agotar el número máximo de intentos por lote. Acepta de forma determinista cuando $f(s') \leq f(s) + T$ y devuelve el promedio de las aceptadas.

```
static Batch batch(Route current, Route best, double
    temperature, Parameters parameters,
    Random random, long evaluated, CostListener
    listener) {
    double currentCost = current.cost();
    double sum = 0.0;
    int accepted = 0;
    int attempts = 0;
    while (accepted < parameters.batchSize() && attempts <
        parameters.maxBatchAttempts()) {
        Route candidate = current.neighbor(random);
        attempts++;
        double candidateCost = candidate.cost();
        if (candidateCost <= currentCost + temperature) {
            current = candidate;
            currentCost = candidateCost;
            accepted++;
            sum += currentCost;
            if (currentCost < best.cost()) {
                best = current;
            }
        }
    }
    double average = accepted == 0 ?
        Double.POSITIVE_INFINITY : sum / accepted;
    return new Batch(average, current, best, attempts,
        accepted);
}
```

Code 1. Método `-NoValue-` para calcular lotes.

4.4.2. Umbrales

Para cada temperatura, se repiten lotes hasta que ya no se mejoren. Se considera que en ese momento se alcanzó el equilibrio térmico. Enfria $T *= \varphi$ y termina al alcanzar ε o agotar el número total de intentos.

```
static Batch batch(Route current, Route best, double
    temperature, Parameters parameters,
    Random random, long evaluated, CostListener
    listener) {
    double currentCost = current.cost();
```

```
double sum = 0.0;
int accepted = 0;
int attempts = 0;
while (accepted < parameters.batchSize() && attempts <
    parameters.maxBatchAttempts()) {
    Route candidate = current.neighbor(random);
    attempts++;
    double candidateCost = candidate.cost();
    if (candidateCost <= currentCost + temperature) {
        current = candidate;
        currentCost = candidateCost;
        accepted++;
        sum += currentCost;
        if (currentCost < best.cost()) {
            best = current;
        }
    }
}
double average = accepted == 0 ?
    Double.POSITIVE_INFINITY : sum / accepted;
return new Batch(average, current, best, attempts,
    accepted);
}
```

Code 2. Ciclo de `-NoValue-`.

4.4.3. Temperatura inicial

Calcula el doble o la mitad de T hasta llegar la aceptación objetivo, y luego llama a la búsqueda binaria.

4.4.4. Porcentaje de aceptación

Camina L pasos una temperatura fija y mide qué fracción fue aceptada.

4.4.5. Búsqueda binaria para la temperatura

Encuentra la temperatura cuya fracción de aceptación iguala el objetivo.

4.5. Análisis con Python

Adicionalmente, se agregan en el proyecto dos *scripts* realizados en Python con el fin de analizar los resultados de las ejecuciones y generar las gráficas mostradas más adelante.

4.5.1. `-NoValue-`

Toma uno o varios CSV de barrido de semillas y por cada uno filtra las corridas factibles, usando el mismo umbral que la propiedad de factibilidad $\Rightarrow f(P) \leq 1$: (`FEASIBLE = 1.0`), y reporta qué fracción de las corridas lo cumplió. Además calcula estadísticas de resumen sobre las factibles: mejor, mediana, promedio, peor y desviación estándar. Genera un histograma de los costos factibles.

4.5.2. `-NoValue-`

Grafica la traza de una única ejecución. Toma el CSV `evaluations,cost` que produce `CostListener` en modo verboso como una curva, que genera una gráfica de costo aceptado contra número de evaluaciones.

5. DETALLES DE IMPLEMENTACIÓN

- **Lenguaje y compilación:** Java 21, compilado y empaquetado con Maven 3.8.7. Las pruebas usan JUnit 5.11.4; la lectura de la base de datos usa `sqlite-jdbc` 3.50.1.0. El empaquetado produce un *jar* ejecutable autocontenido mediante `maven-shade-plugin`.

- **Reproducibilidad:** una única instancia de `java.util.Random`, con una semilla explícita, se pasa por referencia. Ninguna estructura cuyo orden de iteración dependa del *hash code* interviene en el cálculo, de modo que la misma semilla produce exactamente los mismos resultados en cualquier ejecución.
- **Evaluación incremental de costo:** el costo de un vecino no se recalcula desde cero. Como una reversión de segmento altera a lo más dos aristas del camino, el nuevo costo se obtiene sumando al costo actual la diferencia entre esas dos aristas antes y después del movimiento, dividida entre el normalizador.
- **Paralelización:** el barrido de semillas independientes (múltiples ejecuciones del mismo algoritmo sobre la misma instancia, cada una con su propia semilla) se ejecuta sobre un *thread pool* de tamaño configurable, con una escritura sincronizada a un único archivo de resultados para no perder ejecuciones si el proceso se interrumpe a la mitad.
- **Terminación:** además del criterio de enfriamiento $T \leq \varepsilon$, se añadió una cota global de intentos y una cota por lotes. Si la cota global es superada, el algoritmo termina a pesar de no haber llegado a $T \leq \varepsilon$. El motivo de detención es reportado al usuario.

6. RESULTADOS EXPERIMENTALES.

Las pruebas se ejecutaron en un equipo utilizando hasta 24 hilos y 4GB de RAM dedicados. Los barridos de semillas siempre se hacen con un total de 1000 semillas. Para todos los experimentos, se utilizaron dos instancias: una de 40 ciudades (`input-40.tsp`) y otra de 150 ciudades (`input-150.tsp`). Los parámetros del archivo de configuración de propiedades fueron modificados a mano hasta obtener los mejores resultados a ojo.

Las instancias en cuestión son las siguientes:

input-40.tsp

1, 2, 3, 4, 5, 6, 7, 75, 163, 164, 165, 168, 172, 244, 327, 329, 331, 332, 333, 489, 490, 491, 492, 493, 496, 652, 653, 654, 656, 657, 815, 816, 817, 820, 978, 979, 980, 981, 982, 984

input-150.tsp

1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 14, 16, 17, 19, 20, 22, 23, 25, 26, 27, 74, 75, 77, 163, 164, 165, 166, 167, 168, 169, 171, 172, 173, 174, 176, 179, 181, 182, 183, 184, 185, 186, 187, 244, 297, 326, 327, 328, 329, 330, 331, 332, 333, 334, 336, 339, 340, 343, 344, 345, 346, 347, 349, 350, 351, 352, 353, 444, 483, 489, 490, 491, 492, 493, 494, 495, 496, 499, 500, 501, 502, 504, 505, 507, 508, 509, 510, 511, 512, 520, 652, 653, 654, 655, 656, 657, 658, 660, 661, 662, 663, 665, 666, 667, 668, 670, 671, 673, 674, 675, 676, 678, 815, 816, 817, 818, 819, 820, 821, 822, 823, 825, 826, 828, 829, 832, 837, 839, 840, 978, 979, 980, 981, 982, 984, 985, 986, 988, 990, 991, 995, 999, 1001, 1003, 1004, 1037, 1038, 1073, 1075

6.1. Permutaciones

Previo a la fase de experimentación, para probar que el programa fuera correcto y antes de implementar la heurística se programó la resolución exhaustiva mediante el cálculo de todas las permutaciones de una instancia. Para ello se tomaron instancias pequeñas, en específico una de 4 ciudades (`input-4.tsp`) que fue contrastada a mano, una de 8 (`input-8.tsp`) y una de diez (`input-10.tsp`).

input-4.tsp

163, 164, 165, 168

Salida exhaustiva para input-4.tsp

cities = 4
maximum = 3321800.4668155443
feasible = false
cost = 2948746.4821875407
164, 165, 163, 168

input-8.tsp

6, 7, 75, 163, 164, 165, 168, 172

Salida exhaustiva para input-8.tsp

cities = 8
maximum = 4911381.086669923
feasible = true
cost = 0.8955327145822496
75, 165, 172, 168, 163, 7, 6, 164

input-10.tsp

6, 7, 75, 163, 164, 165, 168, 172, 244, 327

Salida exhaustiva para input-10.tsp

cities = 10
maximum = 4911381.086669923
feasible = true
cost = 0.6902065791544622
327, 164, 244, 75, 165, 6, 7, 163, 172, 168

La búsqueda exhaustiva enumera $4! = 24$, $8! = 40\,320$ y $10! = 3\,628\,800$ órdenes respectivamente, y resuelve las tres instancias en menos de medio segundo cada una: 0.24 s, 0.27 s y 0.47 s, midiendo el proceso completo, incluyendo el arranque de la máquina virtual de Java y la carga de la base de datos.

El resultado más informativo es el de `input-4.tsp`: su mejor permutación sigue siendo **infactible**, con un costo de 2.95×10^6 . La subgráfica inducida por esas cuatro ciudades no contiene ningún camino hamiltoniano, y al enumerar las 24 permutaciones se verifica de manera exhaustiva que no existe solución factible alguna. Las otras dos instancias sí admiten camino hamiltoniano. `input-10.tsp` resulta particularmente útil como referencia: su óptimo verdadero, $f(P^*) = 0.6902065791544622$, es el único valor con el que se puede medir de forma exacta qué tan lejos del óptimo queda la heurística, ya que con instancias más grandes la enumeración deja de ser viable.

6.2. Sin búsqueda binaria para la temperatura

La primera implementación de la heurística se llevó a cabo sin búsqueda binaria para la temperatura. Tras correr las instancias de 40 y 150 ciudades, estos fueron los resultados:

40 ciudades. Parámetros: $T_0 = 1000$, $\varphi = 0.99$, $L = 4000$, $\varepsilon = 10^{-5}$, cota de intentos por lote = 6000, cota global = 2.5×10^8 , $P = 0.95$, `searchTemperature = false`; semilla maestra `-5926462348639063530`. Las 1000 corridas tomaron **238.7 s** en total (4.19 corridas/s). El mejor costo fue **0.26391174151846303**, alcanzado por la semilla `5507508804058885315`.

Factibles	973/1000 (97.3%)
Mejor	0.26391174
Mediana	0.29598732
Promedio	0.29486956
Peor	0.44054794
Desviación estándar	0.01355539

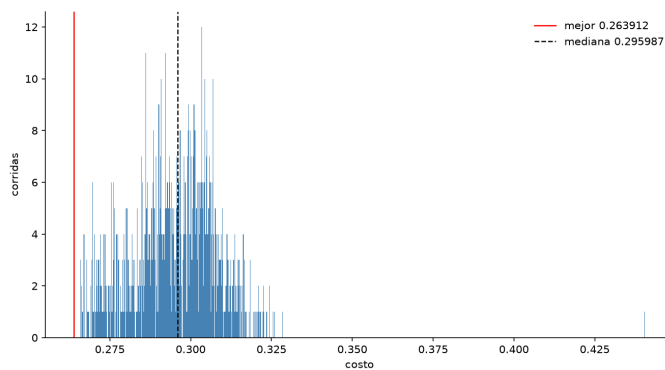


Figure 1. Histograma de 1000 corridas para 40 ciudades sin búsqueda binaria

El histograma generado parece tener una distribución normal.

150 ciudades. Mismos parámetros y misma semilla maestra. Las 1000 corridas tomaron **851.7 s** en total (1.17 corridas/s). El mejor costo fue **0.17462412491928164**, alcanzado por la semilla **-3342339423981252473**.

Factibles	2/1000 (0.2%)
Mejor	0.17462412
Mediana	0.17995192
Promedio	0.17995192
Peor	0.18527972
Desviación estándar	0.00753465

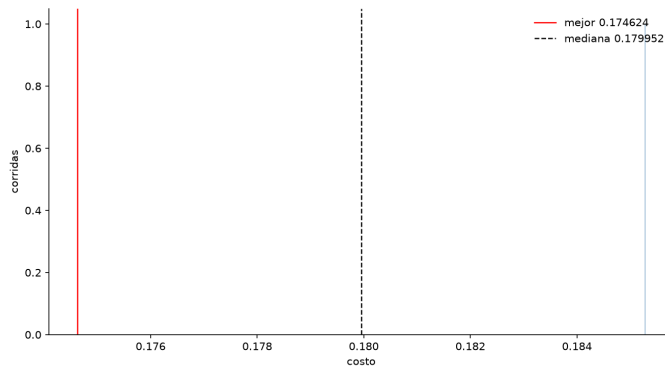


Figure 2. Histograma de 1000 corridas para 150 ciudades sin búsqueda binaria

Para que el histograma tenga sentido visual, sólo toma en cuenta soluciones factibles. Para esta instancia, tan sólo dos soluciones lograron factibilidad, y son precisamente la mejor, marcada en rojo, y la peor, marcada en azul.

Como se puede observar, la instancia de 40 ciudades tuvo un *performance* decente. No obstante, la instancia de 150 ciudades tuvo un porcentaje de factibilidad en sus soluciones finales que no es aceptable. Vale la pena señalar que las estadísticas de la instancia de 150 ciudades se calculan sobre únicamente dos corridas factibles, por lo que su mediana, promedio y desviación estándar no describen el comportamiento del algoritmo, sino solo esas dos corridas. Cuando menos este resultado nos deja ver que en efecto la heurística permite soluciones infactibles.

6.3. Con búsqueda binaria para la temperatura

Después, se probó activando la búsqueda binaria para la temperatura con el objetivo de comparar las soluciones obtenidas cuando se aplica esta técnica.

40 ciudades. Parámetros idénticos a los del experimento anterior, con la única diferencia de `searchTemperature = true`; misma semilla maestra **-5926462348639063530**. Las 1000 corridas tomaron **319.3 s** en total (3.13 corridas/s). El mejor costo fue **0.26381952559689636**, alcanzado por la semilla **811391249151273594**.

Factibles	1000/1000 (100.0%)
Mejor	0.26381953
Mediana	0.29370696
Promedio	0.29242460
Peor	0.33077813
Desviación estándar	0.01319523

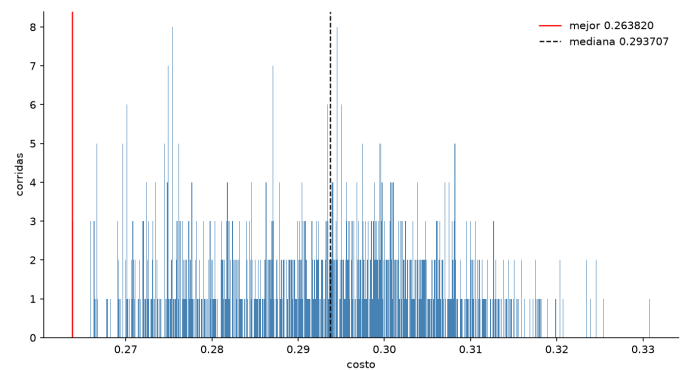


Figure 3. Histograma de 1000 corridas para 40 ciudades con búsqueda binaria

El histograma revelado es en realidad muy parecido al del experimento anterior. Luce más ancho porque la peor solución ya no se aleja tanto de la media. Cabe destacar que tanto la mediana como la mejor solución mejoraron ligeramente. De hecho, la mejor solución es la misma que la que se verá en el siguiente experimento.

150 ciudades. Mismos parámetros y misma semilla maestra. Las 1000 corridas tomaron **1051.0 s** en total (0.95 corridas/s). El mejor costo fue **0.14246516817199148**, alcanzado por la semilla **-8843239329432448181**.

Factibles	769/1000 (76.9%)
Mejor	0.14246517
Mediana	0.17449250
Promedio	0.17502456
Peor	0.50538887
Desviación estándar	0.01524970

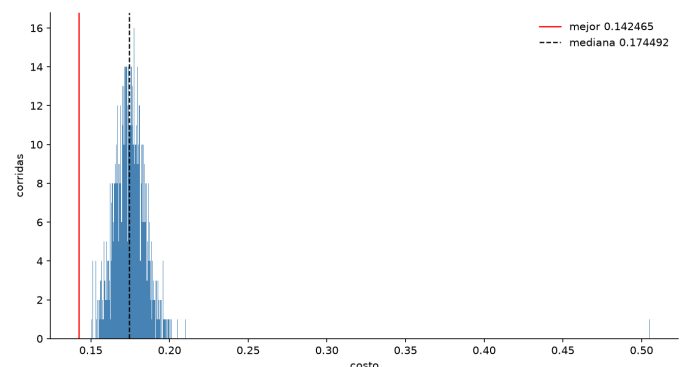


Figure 4. Histograma de 1000 corridas para 40 ciudades con búsqueda binaria

Ahora en ambos casos logramos una distribución normal.

La búsqueda binaria aumentó significativamente el porcentaje de factibilidad: de 97.3% a 100% en la instancia de 40 ciudades, y de 0.2% a 76.9% en la de 150. Es indispensable tener un alto porcentaje de factibilidad; al menos un 80%. De otra manera, estamos generando soluciones inexistentes y que en la práctica no sirven de nada. El costo de esa confiabilidad es tiempo: las mismas 1000 corridas pasaron de 238.7 s a 319.3 s en la instancia de 40 ciudades, y de 851.7 s a 1051.0 s en la de 150, ya que calcular T_0 consume evaluaciones antes de que empiece el enfriamiento propiamente dicho.

6.4. 2-opt

Finalmente se sustituyó el operador de vecindad por la reversión de segmento descrita en la Sección 3, y se reajustaron los parámetros del enfriamiento.

40 ciudades. Parámetros: $T_0 = 500000$, $\varphi = 0.995$, $L = 4000$, $\varepsilon = 10^{-9}$, cota de intentos por lote = 6000, cota global = 10^8 , $P = 0.993$, `searchTemperature = true`; semilla maestra `-1417975048879117167`, 24 hilos. Las 1000 corridas tomaron **831.0 s** en total (1.20 corridas/s). El mejor costo fue **0.26381952559689636**, alcanzado por la semilla `130071593650909319`.

Factibles	1000/1000 (100.0%)
Mejor	0.26381953
Mediana	0.26389366
Promedio	0.26446397
Peor	0.26590735
Desviación estándar	0.00081647

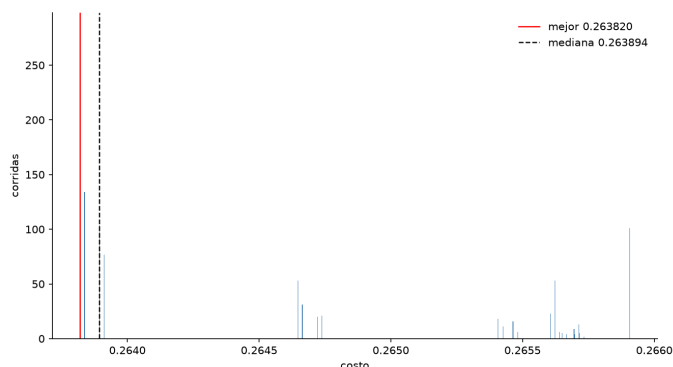


Figure 5. Histograma de 1000 corridas para 40 ciudades con 2-opt

150 ciudades. Mismos parámetros y misma semilla maestra, con 20 hilos. Las 1000 corridas tomaron **2580.0 s** en total (0.39 corridas/s). El mejor costo fue **0.13623807950353167**, alcanzado por la semilla `6091304898049588117`.

Factibles	1000/1000 (100.0%)
Mejor	0.13623808
Mediana	0.13726826
Promedio	0.13748888
Peor	0.14218576
Desviación estándar	0.00096541

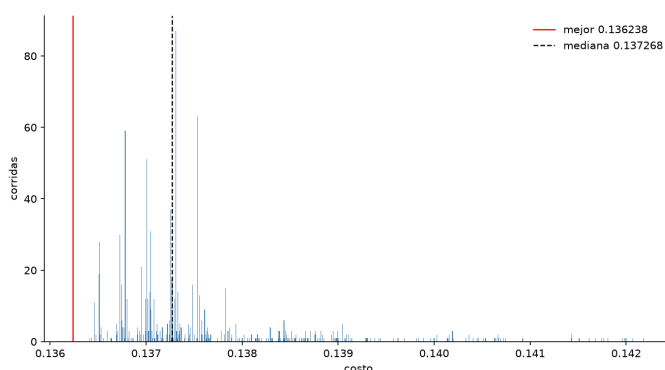


Figure 6. Histograma de 1000 corridas para 150 ciudades con 2-opt

Pareciera que ambos histogramas tienen datos faltantes, pero en realidad lo que sucede es que muchas de las soluciones producidas a partir de distintas semillas convergieron en exactamente los mismos resultados, particularmente en la instancia de tamaño 40.

La calidad de las soluciones mejora tremendamente cuando aplicamos la optimización. Por primera vez logramos costos inferiores a .14. La desviación estándar es el indicador más claro del cambio: pasó de 0.01319523 a 0.00081647 en la instancia de 40 ciudades, y de 0.01524970 a 0.00096541 en la de 150, un orden de magnitud menos en ambos casos. Ya no se trata solo de que la mejor corrida sea mejor, sino de que la peor corrida de este experimento (0.26590735 y 0.14218576 respectivamente) es comparable con la *mejor* corrida de los experimentos anteriores.

Para la instancia de 40 ciudades, el mejor costo logrado es de 0.26381952559689636, en tan solo 3.7 segundos por ejecución individual. En el barrido de 1000 semillas, el número de semillas diferentes que reprodujeron este resultado fue de 284, lo que representa el 28.4% del total. Esto nos lleva a sospechar fuertemente que es muy probable que hayamos encontrado el óptimo de entre todo el espacio de búsqueda. Aún así, resulta imposible asegurarlo sin verificar exhaustivamente que no existe una mejor solución.

En la instancia de 150 ciudades, el mejor resultado encontrado no es tan común como en la de 40 ciudades. En el barrido de 1000 semillas, tan sólo 5 lograron alcanzar ese resultado. De todas formas, en el histograma se observa que la media de soluciones es de 0.13748888, a una distancia de apenas 0.9% del mejor costo encontrado.

6.5. Costo vs Temperatura

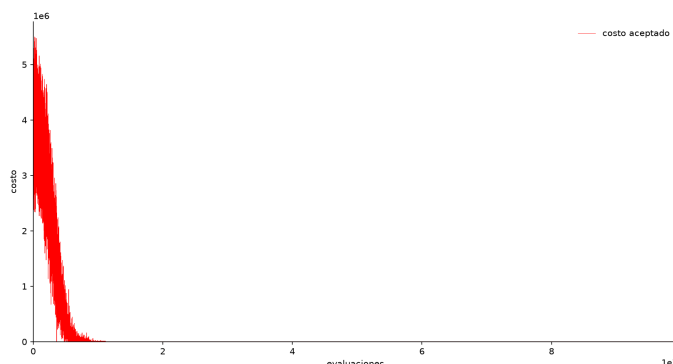


Figure 7. Gráfica de costos aceptados vs temperatura para 40 ciudades

En la de 150 ciudades se aprecia de mejor manera cómo es que la curva al inicio sube ligeramente para después caer, y conforme cae, el rango aceptado se va estrechando.

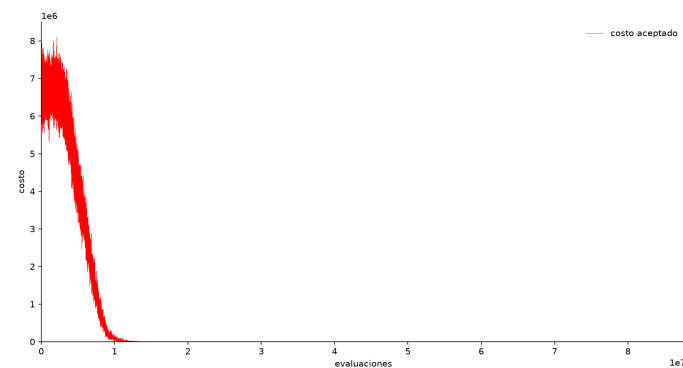


Figure 8. Gráfica de costos aceptados vs temperatura para 150 ciudades

Para observar de forma gráfica el comportamiento de la heurística, se realizaron gráficas de los costos aceptados en función de la temperatura. Guardan cierta similitud con las encontradas en la literatura, aunque la curva que generan es bastante empinada. Podemos ver que al inicio sucede la etapa de exploración, donde se permite que las soluciones empeoren, para luego llegar a valores muy bajo y explotar las mejores soluciones encontradas. La escala ni siquiera nos permite apreciar la línea roja representando el costo.

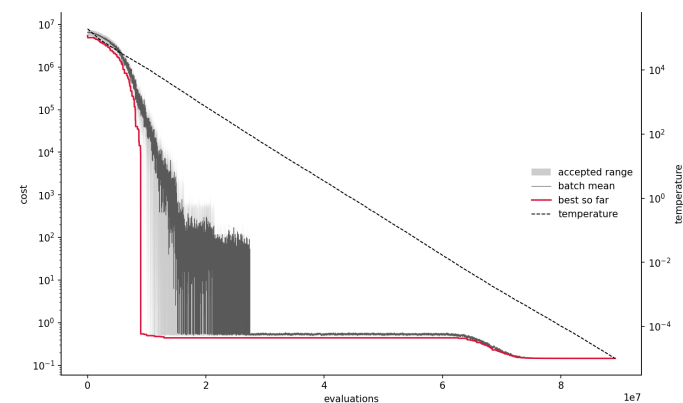


Figure 9. Evolución del costo aceptado por lote y de la temperatura a lo largo de una corrida completa sobre 150 ciudades

También se graficó el análisis de costos contra evaluaciones. Este análisis toma en cuenta la media de costo por lote, donde se ve de forma clara cómo al inicio la factibilidad de las soluciones no es buena, y llega un punto en el que la heurística descarta por completo costos menores a 1. Ya más hacia el final, vemos que hay una bajada suave a costos aún menores, permitidos por tener una ϵ de terminación muy cercana a cero.

6.6. Mapas

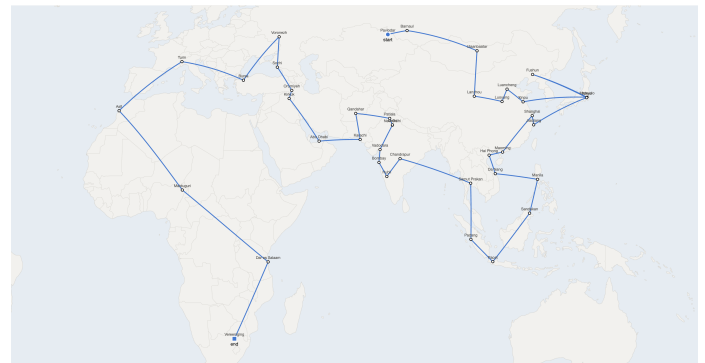


Figure 10. Mapa de la mejor ruta encontrada para 40 ciudades

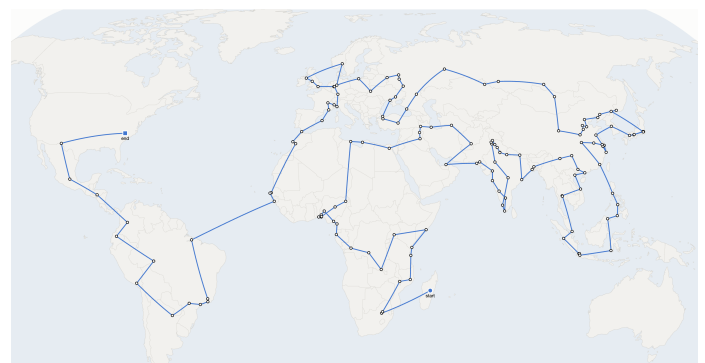


Figure 11. Mapa de la mejor ruta encontrada para 40 ciudades

Por último, la visualización en mapa de la trayectoria entre ciudades nos deja ver que geoméricamente, las mejores soluciones obtenidas tienen sentido. Existen cruces mínimos entre aristas, y es posible que aquellos que sí se presentan sean imposibles de eliminar en esa configuración debido a las características específicas de las instancias y las conexiones inexistentes entre ciudades.

7. CONCLUSIONES

Los resultados presentados mantuvieron los mismos parámetros de tamaño de lote, temperatura inicial, final, factor de enfriamiento, aceptación objetivo y número de intentos permitidos. Para hacer un análisis completo, se deberían de haber reportado las diferencias halladas entre estos parámetros. Aquellos que se usaron para hallar las soluciones fueron elegidos porque en la prueba y error, resultaron mejores. En general, factores como la tasa de enfriamiento producen mejores soluciones mientras más cercana a 1 sea. Sin embargo, este comportamiento no es una regla; había veces donde las soluciones no mejoraban, o incluso empeoraban. Esto sucedía igual para el resto de parámetros.

En cuanto a lo sí reportado, vemos que la implementación de la búsqueda binaria fue fundamental para lograr resultados confiables. Para la mejora de calidad en los resultados en sí, lo que resultó útil fue 2-opt. Es importante revisar resultados con muestras lo suficientemente grandes y con distintos tamaños de instancia para detectar anomalías y patrones en el comportamiento.

Conclusiones personales

Siendo sincera, titubeé acerca de dejar inscrita esta materia. No tenía confianza en mí misma para llegar a soluciones decentes. Este proyecto ha probado lo contrario; terminé dedicándole muchas más horas de lo que tenía planeado. Lo mejor de todo es que conforme más avanzaba con el mismo, más divertido resultaba ver mis resultados e intentar mejorarlos. Hacía tiempo no me resultaba satisfactorio hacer un proyecto académico.

Sin duda uno de los componentes fundamentales para evitar caer en la frustración durante el desarrollo fueron mis compañeros de grupo. Una vez con la heurística implementada, surgió una carrera por ver quién lograba mejores resultados. Esta competencia en ningún momento fue destructiva. Al contrario, se fomentó la mejora y la colaboración continua. Esta fue una pequeña muestra de cómo la colaboración científica da sus frutos, y sin duda me invita a querer repetir la dinámica en el futuro.